

The Automation Playbook Workbook

One workbook that becomes your one-page Automation Playbook — the unit is always the **task**, never the tool. Fill it as the course goes; by the capstone the six tabs **are** your Playbook. The gray rows are the worked example — the 60-person firm's expense report coding task — write your own underneath.

The automation DRAFTS. A named human signs before anything is posted, paid, sent, or filed — you never automate the signature. Anything that touches money, compliance, or an action you can't take back stays a draft a person signs, every time, whatever the score.

1. **Score** your candidate tasks on five plain questions and read the verdict — Module 1.
2. **Choose** the lightest level of the ladder that does the job — Module 2.
3. **Author** the SOP your automation runs on, and point the SOP-Execution Agent at it — Module 3.
4. **Launch** the smallest safe version, with the human's sign-off designed in — Module 4.
5. **Evaluate** it: test the exceptions, arm the pivot switch, measure baseline vs after — Module 5.

S · SCORE

Automatability Scorecard

C · CHOOSE

Approach Pick

A · AUTHOR

Task SOP + Agent

L · LAUNCH

Launch Card

E · EVALUATE

Test / Rollback / Yield

Your Starting Snapshot — the task you’re bringing

You arrive from Tasks, Not Jobs with one task you already decided to automate — your First Move.

Field	Your First Move	Example — the firm's expense report coding task
The task (verb + noun)		<i>Code incoming employee expense reports to the right category</i>
Who does it today		<i>The bookkeeper, in Finance — about 5 hours a week</i>
Who approves its output today (the signer)		<i>The controller — signs the coding before it's reimbursed</i>
What the task touches (data class)		<i>Reimbursements + expense policy → money · Confidential</i>
Why you chose it as your First Move		<i>Execution work, high time-drain, decided 'automate' in Tasks, Not Jobs</i>

The Automatability Scorecard

First three columns: higher is more automatable. Last three (weird / judgment / systems): higher pushes AWAY from automation. Verdict bands are Build now / Not yet / Keep it human — never a traffic light. The band is a floor, not a ceiling.

Task (never the tool)	Data availability	Rule-clarity	Exception rate	Judgment content	Integration surface	Verdict band	Flag	Pick
<i>Code employee expense reports to expense categories</i>	<i>High</i>	<i>High</i>	<i>Low-ish</i>	<i>Low (on coding)</i>	<i>Moderate</i>	<i>Build now</i>	<i>Touches money → drafts only, controller signs</i>	◀
<i>Kick off a new client (intake)</i>	<i>Moderate</i>	<i>Low (not written down)</i>	<i>Mod-High</i>	<i>Moderate</i>	<i>High</i>	<i>Not yet</i>	<i>Author it, narrow scope, re-score</i>	
<i>Review which overdue accounts go to collections</i>	<i>High</i>	<i>Low-Mod</i>	<i>Moderate</i>	<i>High (whole task)</i>	<i>Moderate</i>	<i>Keep it human</i>	<i>Judgment line — a decision</i>	

The Approach Pick

Four levels, lightest first: 1 Prompt/agent recipe · 2 No-code workflow · 3 Connected integration · 4 Custom build. Start on the lowest level that does the job.

Task	Level (start lowest that works)	Why this level	What a later climb would take (the trigger)
Code employee expense reports to expense categories	1 — Prompt / agent recipe	Structured, repeatable, feeds one system; the SOP-Execution Agent drafts, the controller signs. A money-touch does NOT push it up.	Climb to a 2 — No-code workflow only if volume roughly triples, a second entity needs it, or manual copy-paste becomes the bottleneck.

Your Task SOP

You can't automate what you can't write down. Point the SOP-Execution Agent at this; it drafts from these steps, a named human signs. The exceptions become your test cases in Module 5.

SOP part	Your task	Example — expense report coding
Task name		<i>Code incoming employee expense reports to the right category</i>
Trigger (what starts it)		<i>A new employee expense report lands in the AP inbox</i>
Inputs (what the agent reads)		<i>The expense report + receipts; the written expense policy; the category list</i>
Steps (numbered)		<i>1) Read each expense line (merchant, amount, date). 2) Match it to a category. 3) Check it against the policy. 4) Produce a draft coding + a policy flag per line.</i>
Exception rules (if unsure, FLAG — don't guess)		<i>Missing receipt → FLAG. Over-the-limit charge → FLAG. Personal / non-reimbursable item → FLAG.</i>
Sign-off (the named human, before anything is reimbursed)		<i>The controller reviews the draft, edits if needed, and approves the reimbursement. The agent never approves, pays, or issues a reimbursement.</i>

The Launch Card

If a cell is blank, the automation doesn't go live. The approval step is designed in from day one, never bolted on.

Launch Card field	Your automation	Example — expense report coding
Scope (as small as still works)		<i>One merchant category only (travel & meals)</i>
Human approval step (who signs, before what)		<i>The controller signs every draft BEFORE it posts — no auto-post, ever</i>
Boundary (where it runs)		<i>Company accounts + approved AI; drafts only — never a reimbursement or payment</i>
Data class (what it touches)		<i>Confidential — reimbursements + expense policy</i>
Go-live plan (one task, one team, one week)		<i>Week 1, AP + controller only; the rest of the team is told what it is and isn't</i>
Fallback if it misbehaves (the pivot switch)		<i>Turn the agent off; the bookkeeper codes manually — the way it always worked</i>

Test · Rollback · Yield

Test the exceptions your scorecard flagged. Every automation gets a pivot switch. The yield is a labeled estimate until the 90-day checkpoint proves it — hand the honest read to Signal vs Noise.

Field	Your automation	Example — expense report coding (numbers are a labeled synthetic example)
Test cases (MUST include the flagged exceptions)		<i>Missing receipt · over-the-limit charge · personal item · over-threshold — plus ordinary ones</i>
Pivot switch (retune → redirect → stop) + who flips it		<i>Controller flips it: retune the SOP, redirect odd cases to manual, or stop and code by hand</i>
Manual fallback (the way it always worked)		<i>The bookkeeper codes expense reports by hand — no automation ships without this</i>
Baseline — counted BEFORE (label the number)		<i>~5 hrs/week on the pilot category (measured over 2 weeks) — synthetic</i>
After — measured at the checkpoint		<i>~1.5 hrs/week drafting + review — synthetic</i>
Yield (label it an ESTIMATE until day 90)		<i>~3.5 hrs/week back on this one category (estimate) — no 'average', no benchmark</i>
90-day checkpoint date (on the calendar)		<i>Go-live + 90, on the controller's calendar, baseline pasted into the invite</i>
What this yield funds next (the SCALE loop)		<i>Automation #2 — the proven hours pay for the next task; no big-bang spend</i>